

SQL Explanation Grader — Benchmark Report

Run ID: v2_dev | Evaluated At: September 28, 2026 - 16:24:03 | Source File: v2_dev.jsonl

PASS RATE	TOTAL QUERIES	PASSED (GOOD)	FAILED (BAD)	AVG GEN LATENCY
60.0%	50	30	20	134.80s

Performance Breakdown by SQL Feature

SQL Feature	Total Queries	Passed	Failed	Accuracy
subquery in WHERE	10	3	7	30.0%
INNER JOIN of two tables	9	8	1	88.9%
JOIN of three tables with ORDER BY and LIMIT	8	6	2	75.0%
GROUP BY with HAVING	8	4	4	50.0%
window function such as ROW_NUMBER or RANK	7	4	3	57.1%
simple SELECT with WHERE	5	3	2	60.0%
LEFT JOIN with IS NULL filter	3	2	1	66.7%

Itemized Test Case Results

Detailed breakdown of query input, generated explanation, and judge critique for each test item.

#1. q071 Feature: <i>subquery in WHERE</i> Domain: <i>hospital</i>	FAIL (Bad) [13.3s]
SQL Query: SELECT name FROM patients WHERE id = (SELECT patient_id FROM appointments WHERE id = 1);	
Generated Explanation: This query finds the name of the patient who has an appointment with a specific doctor. It first looks at the appointments table to find the patient_id for the appointment with id 1. Then, it uses that patient_id to find the patient's name in the patients table.	
Judge Critique / Reason: The explanation does not cover the full query, as it only describes the first part of the subquery and does not explain how the patient's name is found.	

#2. q141 Feature: <i>subquery in WHERE</i> Domain: <i>hospital</i>	FAIL (Bad) [26.2s]
SQL Query: <pre>SELECT * FROM doctors WHERE name IN (SELECT name FROM patients WHERE birth_year > 1990);</pre> Generated Explanation: This query finds all doctors whose names appear in the list of patients born after 1990. It does this by first looking at the patients table to find names of patients born in years greater than 1990. Then, it uses those names to find corresponding doctors in the doctors table. Judge Critique / Reason: <i>The explanation does not cover the JOIN operation, which is used to link the doctors and patients tables. It also does not explain the IN clause correctly.</i>	
#3. q164 Feature: <i>simple SELECT with WHERE</i> Domain: <i>school</i>	PASS (Good) [36.2s]
SQL Query: <pre>SELECT * FROM students WHERE year = 4;</pre> Generated Explanation: This query returns all the information about students who are in their fourth year. It looks at the students table and only includes rows where the year column has the value 4. Judge Critique / Reason: <i>The explanation correctly states what the query returns, covers the WHERE clause, and is clear for a non-technical reader.</i>	
#4. q140 Feature: <i>subquery in WHERE</i> Domain: <i>hospital</i>	PASS (Good) [50.0s]
SQL Query: <pre>SELECT * FROM appointments WHERE fee = (SELECT AVG(fee) FROM appointments);</pre> Generated Explanation: This query selects all columns from the appointments table where the fee is equal to the average fee of all appointments. First, it calculates the average fee by taking the average of the fee column from the appointments table. Then, it looks for appointments where the fee exactly matches this average fee. Judge Critique / Reason: <i>The explanation correctly describes the query's operation, covers all aspects of the query, and uses clear language.</i>	
#5. q082 Feature: <i>subquery in WHERE</i> Domain: <i>school</i>	PASS (Good) [42.8s]
SQL Query: <pre>SELECT students.name, courses.title FROM students JOIN enrollments ON students.id = enrollments.student_id JOIN courses ON enrollments.course_id = courses.id WHERE courses.credits = (SELECT MAX(credits) FROM courses);</pre> Generated Explanation: This query finds students who are enrolled in courses with the highest number of credits. It matches students with their enrollments and the courses they are taking. For each student, it shows their name and the title of the course they are in, but only if that course has the most credits among all courses. Judge Critique / Reason: <i>The explanation correctly describes the query's operation, covers all aspects (joins, filters, grouping, aggregate, ordering), and uses clear language.</i>	

#6. q100 Feature: <i>JOIN of three tables with ORDER BY and LIMIT</i> Domain: <i>hospital</i>	PASS (Good) [50.6s]
SQL Query: <pre>SELECT patients.name, doctors.name, appointments.appt_date, appointments.fee FROM patients JOIN appointments ON patients.id = appointments.patient_id JOIN doctors ON appointments.doctor_id = doctors.id ORDER BY patients.birth_year LIMIT 1;</pre>	
Generated Explanation: This query finds the name of a patient, the name of their doctor, the date of their appointment, and the fee for that appointment. It looks at three tables: patients, appointments, and doctors. The query matches patients with their appointments based on patient ID, and then matches those appointments with the doctors. It then orders all the patients by their birth year and returns the first row, giving us the earliest-born patient's appointment details.	
Judge Critique / Reason: <i>The explanation correctly identifies the selected columns, joins, filters, and ordering, and explains the LIMIT clause clearly.</i>	
#7. q032 Feature: <i>INNER JOIN of two tables</i> Domain: <i>hotel</i>	PASS (Good) [56.5s]
SQL Query: <pre>SELECT rooms.price_per_night FROM rooms INNER JOIN bookings ON rooms.id = bookings.room_id;</pre>	
Generated Explanation: This query finds the price per night of each room that has been booked. It looks at the rooms table and the bookings table, matching them based on the room ID. For each room that has been booked, it returns the price per night of that room.	
Judge Critique / Reason: <i>The explanation correctly identifies the query's operation, covers the join, and explains the purpose clearly.</i>	
#8. q087 Feature: <i>GROUP BY with HAVING</i> Domain: <i>school</i>	FAIL (Bad) [54.0s]
SQL Query: <pre>SELECT title FROM courses GROUP BY title HAVING COUNT(*) >= 2 AND credits > 2;</pre>	
Generated Explanation: This query finds the titles of courses that have at least two enrollments and more than two credits. It groups the courses by their titles and then filters out those groups where the number of enrollments is less than two or the credits are two or fewer.	
Judge Critique / Reason: <i>The query does not correctly describe the HAVING clause. The HAVING condition should be applied to the grouped data, but the explanation does not mention this.</i>	
#9. q088 Feature: <i>GROUP BY with HAVING</i> Domain: <i>school</i>	FAIL (Bad) [50.2s]
SQL Query: <pre>SELECT student_id, grade FROM enrollments GROUP BY student_id HAVING AVG(grade) < 2.0;</pre>	
Generated Explanation: This query finds students who have an average grade less than 2.0 across all their courses. It groups the enrollments by student ID and then filters out those students whose average grade is 2.0 or higher.	
Judge Critique / Reason: <i>It incorrectly states that the query filters out students whose average grade is 2.0 or higher. The HAVING clause is used for filtering aggregated results, not individual rows. The correct interpretation is that it finds students with an average grade less than 2.0.</i>	

#10. q114 Feature: <i>INNER JOIN of two tables</i> Domain: <i>hospital</i>	PASS (Good) [47.4s]
SQL Query: <pre>SELECT appointments.appt_date, patients.name, doctors.name, appointments.fee FROM appointments INNER JOIN doctors ON appointments.doctor_id = doctors.id INNER JOIN patients ON appointments.patient_id = patients.id;</pre>	
Generated Explanation: This query retrieves the appointment date, the patient's name, the doctor's name, and the fee for each appointment. It matches the appointments table with the patients table on the patient ID, and with the doctors table on the doctor ID, ensuring that we get the names of both the patients and doctors involved in each appointment.	
Judge Critique / Reason: <i>The explanation correctly identifies the selected columns, joins, and filters, and explains each component clearly.</i>	
#11. q089 Feature: <i>GROUP BY with HAVING</i> Domain: <i>school</i>	PASS (Good) [46.7s]
SQL Query: <pre>SELECT course_id, COUNT(*) FROM enrollments GROUP BY course_id HAVING COUNT(*) % 2 = 0;</pre>	
Generated Explanation: This query counts how many students are enrolled in each course and then groups the results by course. It only includes courses where the number of students is an even number.	
Judge Critique / Reason: <i>The explanation correctly describes the query's functionality, covers all aspects (select, group by, having), and uses clear language.</i>	
#12. q051 Feature: <i>JOIN of three tables with ORDER BY and LIMIT</i> Domain: <i>bank</i>	FAIL (Bad) [51.5s]
SQL Query: <pre>SELECT accounts.type, transactions.tx_date, clients.name FROM accounts JOIN clients JOIN transactions ON accounts.client_id = clients.id AND accounts.id = transactions.account_id ORDER BY accounts.id LIMIT 1;</pre>	
Generated Explanation: This query shows the type of account, the date of a transaction, and the name of the client for the first account in the list. It looks at three tables: accounts, clients, and transactions. The accounts table is matched with the clients table based on the client_id, and also with the transactions table based on the account_id. The results are ordered by the account ID to ensure the first row is selected, and only that row is returned.	
Judge Critique / Reason: <i>It does not cover every aspect required: it misses the JOIN explanation for clients and the fact that only one row is returned.</i>	
#13. q043 Feature: <i>subquery in WHERE</i> Domain: <i>library</i>	FAIL (Bad) [54.8s]
SQL Query: <pre>SELECT * FROM loans WHERE book_id IN (SELECT id FROM books WHERE year > 2000);</pre>	
Generated Explanation: This query selects all columns from the loans table where the book_id matches a book that was published after the year 2000. It does this by first finding all book IDs from the books table where the year is greater than 2000, and then selecting all loans that have one of those book IDs.	
Judge Critique / Reason: <i>It does not cover every aspect required: it misses explaining the IN clause and the filtering of loans based on the IN condition.</i>	

#14. q101 Feature: <i>JOIN of three tables with ORDER BY and LIMIT</i> Domain: <i>hospital</i>	FAIL (Bad) [60.2s]
SQL Query: <pre>SELECT patients.name, doctors.name, appointments.appt_date, appointments.fee FROM patients JOIN appointments ON patients.id = appointments.patient_id JOIN doctors ON appointments.doctor_id = doctors.id ORDER BY patients.name LIMIT 5;</pre> Generated Explanation: This query shows the names of patients, the names of the doctors they see, the date of their appointment, and how much the appointment cost. It looks at three tables: patients, appointments, and doctors. The query matches patients with their appointments and then matches those appointments with the doctors. It then lists the first five patients in alphabetical order by their names. Judge Critique / Reason: <i>It does not cover every aspect of the query. It misses the ORDER BY clause and the LIMIT 5.</i>	
#15. q125 Feature: <i>JOIN of three tables with ORDER BY and LIMIT</i> Domain: <i>library</i>	PASS (Good) [75.3s]
SQL Query: <pre>SELECT books.title, loans.loan_date, members.name FROM books JOIN loans ON books.id = loans.book_id JOIN members ON loans.member_id = members.id WHERE books.year > 2000 ORDER BY loans.loan_date LIMIT 5;</pre> Generated Explanation: This query finds the title of books published after the year 2000, along with the date when they were borrowed and the name of the borrower. It looks at three tables: books, loans, and members. The books table is matched with the loans table based on the book ID, and the loans table is matched with the members table based on the member ID. Only books published after 2000 are included. The results are ordered by the loan date, and only the first five loans are shown. Judge Critique / Reason: <i>The explanation correctly describes the query's functionality, covers all aspects of the query (joins, filters, grouping, aggregate, ordering, and LIMIT), and uses clear language.</i>	
#16. q126 Feature: <i>window function such as ROW_NUMBER or RANK</i> Domain: <i>library</i>	PASS (Good) [67.4s]
SQL Query: <pre>SELECT RANK() OVER (ORDER BY year DESC) AS year_rank, * FROM books;</pre> Generated Explanation: This query ranks books by their publication year, with the most recent year ranked first. It then returns all columns from the books table for each book, along with a rank number. The rank is calculated by ordering the books in descending order of their year, so the newest book gets a rank of 1. Judge Critique / Reason: <i>The explanation correctly describes the query's functionality, covers the ranking and ordering, and explains the use of RANK(). It also clarifies that the query returns all columns from the books table.</i>	
#17. q045 Feature: <i>subquery in WHERE</i> Domain: <i>library</i>	FAIL (Bad) [65.2s]
SQL Query: <pre>SELECT * FROM books WHERE author IN (SELECT name FROM members WHERE id = 1);</pre> Generated Explanation: This query selects all columns from the books table where the author's name matches one of the names in the members table. Specifically, it looks for authors whose names are the same as the name of the member with id 1. Judge Critique / Reason: <i>The explanation does not cover the WHERE clause correctly. It should state that the query filters books by authors whose names are in the members table, not that it looks for authors with a specific member's name.</i>	

#18. q054 Feature: <i>simple SELECT with WHERE</i> Domain: <i>hospital</i>	PASS (Good) [59.9s]
SQL Query: <pre>SELECT * FROM patients WHERE birth_year > 1990;</pre> Generated Explanation: This query returns all columns from the patients table for people who were born after 1990. It filters the rows in the patients table to include only those where the birth_year is greater than 1990. Judge Critique / Reason: <i>The explanation correctly states that the query returns all columns from the patients table for people born after 1990, covers the WHERE clause, and is clear for a non-technical reader.</i>	
#19. q029 Feature: <i>INNER JOIN of two tables</i> Domain: <i>hospital</i>	PASS (Good) [1049.4s]
SQL Query: <pre>SELECT patients.id, doctors.id, appointments.appt_date, patients.name AS patient_name, doctors.name AS doctor_name FROM patients INNER JOIN appointments ON patients.id = appointments.patient_id INNER JOIN doctors ON appointments.doctor_id = doctors.id;</pre> Generated Explanation: This query matches patients with their appointments and the doctors they see. It shows the patient's ID, the doctor's ID, the appointment date, the patient's name, and the doctor's name. The query looks at three tables: patients, appointments, and doctors. It finds the matching rows by linking patients to appointments based on the patient's ID, and then linking those appointments to doctors based on the doctor's ID. Judge Critique / Reason: <i>The explanation correctly identifies the tables involved, the join conditions, and the selected columns. It also explains the purpose of the query clearly.</i>	
#20. q159 Feature: <i>GROUP BY with HAVING</i> Domain: <i>retail</i>	FAIL (Bad) [1053.4s]
SQL Query: <pre>SELECT name FROM products GROUP BY name HAVING AVG(price) > 10;</pre> Generated Explanation: This query finds the names of products that have an average price greater than 10. It groups the products by their names and then filters out those groups where the average price is not more than 10. Judge Critique / Reason: <i>It incorrectly states that the query filters out groups where the average price is not more than 10. The HAVING clause is used for filtering aggregated results, not individual groups.</i>	
#21. q034 Feature: <i>INNER JOIN of two tables</i> Domain: <i>hotel</i>	PASS (Good) [1066.5s]
SQL Query: <pre>SELECT rooms.room_type FROM rooms INNER JOIN bookings ON rooms.id = bookings.room_id;</pre> Generated Explanation: This query finds the type of room booked by looking at the rooms table and the bookings table. It matches each booking with the corresponding room by checking that the room IDs in both tables match. The result is a list of room types from the rooms table. Judge Critique / Reason: <i>The explanation correctly describes the query's operation, covers the join, and explains the result clearly.</i>	

#22. q074 Feature: <i>subquery in WHERE</i> Domain: <i>hospital</i>	FAIL (Bad) [1071.5s]
SQL Query: <pre>SELECT name FROM patients WHERE birth_year = (SELECT birth_year FROM patients WHERE id = 1);</pre> Generated Explanation: This query selects the name of a patient whose birth year matches the birth year of the patient with id 1. It does this by first finding the birth year of the patient with id 1 and then looking for any other patient who has the same birth year. Judge Critique / Reason: <i>The explanation does not cover the selection of names, only the process of finding the birth year.</i>	
#23. q132 Feature: <i>INNER JOIN of two tables</i> Domain: <i>bank</i>	PASS (Good) [41.5s]
SQL Query: <pre>SELECT clients.branch, accounts.type FROM clients INNER JOIN accounts ON clients.id = accounts.client_id;</pre> Generated Explanation: This query matches each client with their accounts. It looks at the `clients` table and the `accounts` table, finding rows where the `client_id` in the `accounts` table matches the `id` in the `clients` table. The result shows each client's branch and the type of their accounts. Judge Critique / Reason: <i>The explanation correctly describes the INNER JOIN operation, the tables involved, and the selected columns. It is clear and non-technical.</i>	
#24. q127 Feature: <i>window function such as ROW_NUMBER or RANK</i> Domain: <i>library</i>	FAIL (Bad) [52.5s]
SQL Query: <pre>SELECT ROW_NUMBER() OVER (ORDER BY loan_date) AS loan_num, * FROM loans;</pre> Generated Explanation: This query assigns a unique number to each loan based on the order of the loan dates. It starts numbering from 1 and goes up, so the first loan in the order gets a loan number of 1, the second gets 2, and so on. It then returns all the details of each loan, including the loan number, the book ID, the member ID, the loan date, and whether the book has been returned. Judge Critique / Reason: <i>The explanation does not cover every aspect required: it misses the fact that the query returns all columns and does not just the loan details.</i>	
#25. q142 Feature: <i>subquery in WHERE</i> Domain: <i>hospital</i>	FAIL (Bad) [60.8s]
SQL Query: <pre>SELECT * FROM patients WHERE id IN (SELECT patient_id FROM appointments WHERE appt_date LIKE '%2023%');</pre> Generated Explanation: This query selects all information about patients who have appointments in the year 2023. It does this by first finding the patient IDs of patients who have appointments on dates that include '2023' in the date string. Then, it retrieves all details about these patients from the patients table. Judge Critique / Reason: <i>The explanation does not cover the WHERE clause correctly. It should state that the query filters patients based on appointments with appt_date containing '2023', not just any appointments in 2023.</i>	

#26. q048 Feature: <i>JOIN of three tables with ORDER BY and LIMIT</i> Domain: <i>bank</i>	PASS (Good) [71.2s]
SQL Query: <pre>SELECT clients.name, accounts.type, transactions.tx_date FROM clients JOIN accounts JOIN transactions ON clients.id = accounts.client_id AND accounts.id = transactions.account_id ORDER BY clients.id LIMIT 1;</pre>	
Generated Explanation: This query shows the name of each client, the type of their account, and the date of a transaction related to that account. It looks at three tables: clients, accounts, and transactions. First, it matches each client with their accounts by using the client's ID. Then, it finds transactions related to those accounts. The results are shown in order of the client's ID, and only the first row is returned.	
Judge Critique / Reason: <i>The explanation correctly identifies the selected columns, the joins, the filters, the grouping by client ID, the ordering, and the LIMIT. It also explains each part clearly.</i>	
#27. q097 Feature: <i>JOIN of three tables with ORDER BY and LIMIT</i> Domain: <i>hospital</i>	PASS (Good) [70.4s]
SQL Query: <pre>SELECT patients.name, doctors.name, appointments.appt_date FROM patients JOIN appointments ON patients.id = appointments.patient_id JOIN doctors ON appointments.doctor_id = doctors.id ORDER BY appointments.appt_date LIMIT 1;</pre>	
Generated Explanation: This query finds the name of a patient, the name of their doctor, and the date of their appointment, starting from the earliest appointment. It looks at three tables: patients, appointments, and doctors. The query matches patients with their appointments and then matches those appointments with the doctors. It then orders all the appointments by the date they are scheduled and picks the one that is first, which is the earliest appointment.	
Judge Critique / Reason: <i>The explanation correctly describes the query's purpose, covers all aspects (joins, filters, grouping, aggregate, ordering, and LIMIT), and uses clear language.</i>	
#28. q058 Feature: <i>simple SELECT with WHERE</i> Domain: <i>library</i>	FAIL (Bad) [60.3s]
SQL Query: <pre>SELECT * FROM loans WHERE returned = 0;</pre>	
Generated Explanation: This query retrieves all information about books that have not been returned. It looks at the loans table and selects every row where the 'returned' column is 0, meaning the book is currently out on loan.	
Judge Critique / Reason: <i>The explanation does not cover every aspect required: it misses the fact that the query returns all columns, not just the 'returned' column.</i>	
#29. q147 Feature: <i>GROUP BY with HAVING</i> Domain: <i>school</i>	PASS (Good) [55.4s]
SQL Query: <pre>SELECT student_id, COUNT(*) FROM enrollments GROUP BY student_id HAVING COUNT(*) = 2;</pre>	
Generated Explanation: This query counts how many courses each student is enrolled in and returns the student IDs of those who are enrolled in exactly two courses.	
Judge Critique / Reason: <i>The explanation correctly identifies the query's operation, covers all aspects (join, filter, grouping, aggregate, ordering), and uses clear language.</i>	

#30. q110 Feature: <i>JOIN of three tables with ORDER BY and LIMIT</i> Domain: <i>hospital</i>	PASS (Good) [53.2s]
SQL Query: <pre>SELECT doctors.name, patients.name, appointments.appt_date, appointments.fee FROM patients JOIN appointments ON patients.id = appointments.patient_id JOIN doctors ON appointments.doctor_id = doctors.id ORDER BY appointments.appt_date LIMIT 10;</pre>	
Generated Explanation: This query shows the name of a patient, the name of the doctor they saw, the date of their appointment, and how much the appointment cost. It looks at three tables: patients, doctors, and appointments. It matches patients with their appointments and the doctors who saw them. The results are sorted by the appointment date, and only the first ten appointments are shown.	
Judge Critique / Reason: <i>The explanation correctly identifies the selected columns, the join conditions, the order by clause, and the LIMIT. It also accurately describes the query's purpose and structure.</i>	
#31. q092 Feature: <i>INNER JOIN of two tables</i> Domain: <i>retail</i>	PASS (Good) [49.0s]
SQL Query: <pre>SELECT customers.name, orders.quantity, orders.order_date FROM customers INNER JOIN orders ON customers.id = orders.customer_id;</pre>	
Generated Explanation: This query shows the name of each customer, the quantity of products they ordered, and the date of each order. It matches customers with their orders by using the customer ID, ensuring that only customers who have placed orders are included in the results.	
Judge Critique / Reason: <i>The explanation correctly identifies the selected columns, the join condition, and the filtering by customer ID. It also accurately describes the purpose of the query.</i>	
#32. q002 Feature: <i>window function such as ROW_NUMBER or RANK</i> Domain: <i>retail</i>	PASS (Good) [56.1s]
SQL Query: <pre>SELECT ROW_NUMBER() OVER (PARTITION BY category ORDER BY price) AS price_rank FROM products;</pre>	
Generated Explanation: This query calculates a ranking for each product based on its price within each category. It uses a window function to assign a rank to each product in the `products` table, partitioning the data by the `category` column and ordering the products by their `price` in ascending order. The rank is assigned as the `price_rank`, with the lowest price getting a rank of 1 within its category.	
Judge Critique / Reason: <i>The explanation correctly describes the query's operation, including the use of ROW_NUMBER() and PARTITION BY, ORDER BY, and the resulting price_rank column.</i>	
#33. q047 Feature: <i>LEFT JOIN with IS NULL filter</i> Domain: <i>library</i>	PASS (Good) [76.9s]
SQL Query: <pre>SELECT b.title, m.name, COALESCE(l.loan_date, 'Never') FROM books b LEFT JOIN loans l ON b.id = l.book_id LEFT JOIN members m ON l.member_id = m.id WHERE l.loan_date IS NULL;</pre>	
Generated Explanation: This query finds books that have not been loaned out. It looks at the books table and checks if there are any corresponding entries in the loans table. If a book has been loaned out, the loans table will have an entry with a loan_date. If no such entry exists (meaning the book has never been loaned), the query still includes the book by using a LEFT JOIN. For these books, the loan_date is shown as 'Never'. It also matches these books with members who have borrowed them, listing the member's name if they have borrowed the book. If a book has never been loaned, the member's name will be 'Never' as well.	
Judge Critique / Reason: <i>The explanation correctly describes the query's operation, covers all aspects of the query, and uses clear language.</i>	

#34. q009 Feature: <i>window function such as ROW_NUMBER or RANK</i> Domain: <i>bank</i>	FAIL (Bad) [69.3s]
SQL Query: <pre>SELECT *, RANK() OVER (ORDER BY balance DESC) FROM accounts;</pre> Generated Explanation: This query selects all columns from the accounts table and adds a ranking of each account based on its balance, with the highest balance getting a rank of 1. The ranking is calculated by ordering the accounts in descending order of their balance. Judge Critique / Reason: <i>It fails the Clear criterion. The explanation does not clearly describe what the query does, especially the RANK() function and its effect on the output.</i>	
#35. q006 Feature: <i>LEFT JOIN with IS NULL filter</i> Domain: <i>library</i>	PASS (Good) [73.5s]
SQL Query: <pre>SELECT * FROM books LEFT JOIN loans ON books.id = loans.book_id WHERE loans.returned IS NULL;</pre> Generated Explanation: This query returns all columns from the books table and the corresponding columns from the loans table, but only includes books that have not been returned. It looks at every book in the books table and checks if there is a matching loan in the loans table where the book has not been returned yet (meaning the returned column is NULL). If there is no matching loan, the book information is still included, but the loan columns will show NULL. Judge Critique / Reason: <i>The explanation correctly describes the query's operation, covers all aspects including the join, filter, and the inclusion of NULL values, and is clear for a non-technical reader.</i>	
#36. q161 Feature: <i>GROUP BY with HAVING</i> Domain: <i>retail</i>	FAIL (Bad) [64.7s]
SQL Query: <pre>SELECT COUNT(*) FROM orders GROUP BY customer_id HAVING SUM(quantity) > 100;</pre> Generated Explanation: This query counts the number of orders for each customer, but only includes customers who have ordered more than 100 items in total. It groups the orders by customer and then filters out any groups where the total quantity of items ordered is 100 or less. Judge Critique / Reason: <i>It incorrectly states that the query counts the number of orders for each customer. The correct operation is to count the total number of orders grouped by customer_id. Additionally, the HAVING clause is used to filter groups, not individual rows. The query should be checking the total quantity per customer, not the number of orders.</i>	
#37. q148 Feature: <i>GROUP BY with HAVING</i> Domain: <i>school</i>	PASS (Good) [48.7s]
SQL Query: <pre>SELECT course_id, COUNT(*) FROM enrollments GROUP BY course_id HAVING COUNT(*) = 1;</pre> Generated Explanation: This query counts how many students are enrolled in each course and then filters to show only the courses that have exactly one student enrolled. Judge Critique / Reason: <i>The explanation correctly describes the query's operation, covers all aspects of the query (join, filter, grouping, and aggregate), and uses clear language.</i>	

#38. q046 | Feature: *subquery in WHERE* | Domain: *library*

FAIL (Bad) [53.5s]

SQL Query:

```
SELECT * FROM loans WHERE returned IN (SELECT loan_date FROM loans WHERE book_id = 1);
```

Generated Explanation:

This query looks for all loans where the `returned` field is the same as a `loan\_date` from another loan where the `book\_id` is 1. In other words, it finds all loans that have the same return date as a loan of the same book.

Judge Critique / Reason:

The explanation does not cover the WHERE clause correctly. It should state that the query finds loans where the `returned` date is the same as a `loan\_date` from a loan with `book\_id` 1, not that the `returned` field is the same as a `loan\_date` from another loan with `book\_id` 1.

#39. q068 | Feature: *LEFT JOIN with IS NULL filter* | Domain: *library*

FAIL (Bad) [50.2s]

SQL Query:

```
SELECT * FROM members LEFT JOIN loans ON members.id = loans.member_id WHERE loans.book_id IS NULL;
```

Generated Explanation:

This query shows all members and their loans, but only includes members who have not lent out any books. It looks at the members table and matches it with the loans table based on the member's ID. If a member has no corresponding entry in the loans table (meaning they haven't lent out any books), their information is still included in the result, but the loans part will show no information.

Judge Critique / Reason:

It incorrectly states that the query shows all members and their loans, but only includes members who have not lent out any books. The query actually shows all members with no corresponding loans, which is different.

#40. q156 | Feature: *INNER JOIN of two tables* | Domain: *bank*

PASS (Good) [46.4s]

SQL Query:

```
SELECT clients.id, transactions.amount FROM clients INNER JOIN transactions ON clients.id = transactions.account_id;
```

Generated Explanation:

This query finds the client ID and transaction amount for each transaction. It looks at the clients table and the transactions table, matching them based on the client ID in the clients table and the account ID in the transactions table. Only transactions that have a matching client are included in the result.

Judge Critique / Reason:

The explanation correctly describes the query's operation, including the join condition and the inclusion of matching transactions.

#41. q014 | Feature: *GROUP BY with HAVING* | Domain: *library*

PASS (Good) [55.3s]

SQL Query:

```
SELECT title, AVG(year) FROM books GROUP BY title HAVING AVG(year) > 2010;
```

Generated Explanation:

This query finds the average year of publication for each book title and returns only those titles where the average year is after 2010. It looks at the 'books' table to get the book titles and their publication years, then groups the results by book title to calculate the average year for each one. Only groups where the average year is greater than 2010 are included in the final result.

Judge Critique / Reason:

The explanation correctly describes the query's operation, covers all aspects (select, group by, having), and uses clear language.

#42. q154 Feature: <i>INNER JOIN of two tables</i> Domain: <i>bank</i>	FAIL (Bad) [47.7s]
SQL Query: <pre>SELECT accounts.balance, transactions.tx_date FROM accounts INNER JOIN transactions ON accounts.id = transactions.account_id;</pre> Generated Explanation: This query combines information from the accounts and transactions tables. It looks at each account and finds the corresponding transaction. For every account, it shows the account balance and the date of the most recent transaction. Judge Critique / Reason: <i>It incorrectly states that the query shows the most recent transaction date for each account. The query only returns the balance and the date of the most recent transaction, not all transactions.</i>	
#43. q061 Feature: <i>simple SELECT with WHERE</i> Domain: <i>library</i>	PASS (Good) [46.8s]
SQL Query: <pre>SELECT * FROM loans WHERE book_id = 1;</pre> Generated Explanation: This query retrieves all information about loans for the book with the ID 1. It looks at the loans table and returns every column for rows where the book_id matches 1. Judge Critique / Reason: <i>The explanation correctly states what the query does, covers all aspects of the query, and uses clear language.</i>	
#44. q001 Feature: <i>window function such as ROW_NUMBER or RANK</i> Domain: <i>retail</i>	PASS (Good) [53.9s]
SQL Query: <pre>SELECT RANK() OVER (ORDER BY order_date) AS order_rank FROM orders;</pre> Generated Explanation: This query calculates a rank for each order based on the order date. It uses a window function to assign a rank to each row in the `orders` table, ordering the rows by the `order_date` in ascending order (earliest date first). The rank starts from 1 for the earliest order date and increases for each subsequent order date. Judge Critique / Reason: <i>The explanation correctly describes the query and the use of the RANK() window function.</i>	
#45. q096 Feature: <i>INNER JOIN of two tables</i> Domain: <i>retail</i>	PASS (Good) [48.6s]
SQL Query: <pre>SELECT orders.id, customers.name, products.name, orders.quantity, orders.order_date FROM orders INNER JOIN customers ON orders.customer_id = customers.id INNER JOIN products ON orders.product_id = products.id;</pre> Generated Explanation: This query shows details about each order. It lists the order ID, the customer's name, the product name, the quantity ordered, and the order date. It matches orders with customers and products by their IDs, so even if a customer or product has no orders, they won't be included in the results. Judge Critique / Reason: <i>The explanation correctly describes the query's purpose, covers all aspects of the query (joins, filters, grouping, aggregate, ordering), and uses clear language.</i>	

#46. q012 Feature: <i>window function such as ROW_NUMBER or RANK</i> Domain: <i>bank</i>	FAIL (Bad) [48.6s]
SQL Query: <pre>SELECT *, ROW_NUMBER() OVER (ORDER BY amount) FROM transactions;</pre> Generated Explanation: This query selects all columns from the `transactions` table and adds a row number based on the `amount` column. The row numbers start from 1 and increase as the `amount` values get larger, ordering the transactions by their amount. Judge Critique / Reason: <i>It fails the Clear criterion as it does not explain what the query returns in plain language.</i>	
#47. q021 Feature: <i>window function such as ROW_NUMBER or RANK</i> Domain: <i>retail</i>	PASS (Good) [53.5s]
SQL Query: <pre>SELECT ROW_NUMBER() OVER (PARTITION BY category ORDER BY price) AS row_num, name, price, category FROM products;</pre> Generated Explanation: This query assigns a row number to each product within its category, ordering by price. For each category, the products are ranked starting from 1, with the lowest priced product getting row number 1. The query returns the product name, price, and category for each row, along with the assigned row number. Judge Critique / Reason: <i>The explanation correctly describes the query's operation, including the use of ROW_NUMBER, PARTITION BY, ORDER BY, and the columns returned.</i>	
#48. q083 Feature: <i>subquery in WHERE</i> Domain: <i>school</i>	PASS (Good) [56.0s]
SQL Query: <pre>SELECT students.name, courses.title FROM students JOIN enrollments ON students.id = enrollments.student_id JOIN courses ON enrollments.course_id = courses.id WHERE courses.credits > (SELECT AVG(credits) FROM courses);</pre> Generated Explanation: This query finds students who are enrolled in courses that have more credits than the average number of credits for all courses. It matches students with their enrollments and the courses they are taking. For each student, it looks at the courses they are enrolled in and checks if the course has more credits than the average credits of all courses. If a student is enrolled in such a course, their name and the title of the course are returned. Judge Critique / Reason: <i>The explanation correctly describes the query's operation, covers all aspects including joins, filters, grouping, and aggregate, and uses clear language.</i>	
#49. q109 Feature: <i>JOIN of three tables with ORDER BY and LIMIT</i> Domain: <i>hospital</i>	PASS (Good) [61.3s]
SQL Query: <pre>SELECT patients.name, doctors.name, appointments.appt_date, appointments.fee, appointments.id FROM patients JOIN appointments ON patients.id = appointments.patient_id JOIN doctors ON appointments.doctor_id = doctors.id ORDER BY appointments.appt_date LIMIT 10;</pre> Generated Explanation: This query returns the name of a patient, the name of their doctor, the appointment date, the fee for the appointment, and the appointment ID for the first 10 appointments listed by date. It matches patients with their appointments and doctors, using the patient's ID to link them to their appointment and the appointment's doctor ID to find the doctor's name. The results are sorted by the appointment date, with the earliest appointments appearing first, and only the top 10 are shown. Judge Critique / Reason: <i>The explanation covers all aspects of the query including joins, filters, grouping, aggregate, ordering, and LIMIT. It is clear and non-technical.</i>	

#50. q059 | Feature: *simple SELECT with WHERE* | Domain: *library*

FAIL (Bad) [65.8s]

SQL Query:

```
SELECT * FROM books WHERE author = 'John Doe';
```

Generated Explanation:

This query returns all columns from the books table where the author is John Doe. It looks for books written by John Doe and lists all the details about those books, such as their title, author, and year of publication.

Judge Critique / Reason:

It incorrectly states that all columns are returned, when only the author and potentially other columns are relevant based on the WHERE clause. The query does not specify which columns to return.